Foundations of Backpropagation

[Backpropagation]

$$\frac{\partial L}{\partial W} = \delta \cdot a^T$$

$$\delta^{(l)} = (W^{(l+1)T}\delta^{(l+1)}) \odot \sigma'(z^{(l)})$$

1.0 Content Block

Derivation The gradient descent method involves calculating the derivative of the loss function with respect to the weights of the network. This is normally done using backpropagation. Assuming one output neuron, the squared error function is

2.0 Content Block

where d a L d z L {\displaystyle {\frac {da^{L}}{dz^{L}}}} is a diagonal matrix. These terms are: the derivative of the loss function; the derivatives of the activation functions; and the matrices of weights:

3.0 Content Block

The input net j {\displaystyle {\text{net}}_{j}} to a neuron is the weighted sum of outputs o k {\displaystyle o_{k}} of previous neurons. If the neuron is in the first layer after the input layer, the o k {\displaystyle o_{k}} of the input layer are simply the inputs x k {\displaystyle x_{k}} to the network. The number of input units to the neuron is n {\displaystyle n} . The variable w k j {\displaystyle w_{kj}} denotes the weight between neuron k {\displaystyle k} of the previous layer and neuron j {\displaystyle j} of the current layer.

4.0 Content Block

a j l {\displaystyle a_{j}^{l}} : activation of the j {\displaystyle j} -th node in layer l {\displaystyle l} . In the derivation of backpropagation, other intermediate quantities are used by introducing them as needed below. Bias terms are not treated specially since they correspond to a weight with a fixed input of 1. For backpropagation the specific loss function and activation functions do not matter as long as they and their derivatives can be evaluated efficiently. Traditional activation functions include sigmoid, tanh, ReLU, Swish, Mish,, and many others. The overall network is a combination of function composition and matrix multiplication:

5.0 Content Block

Example loss function Let y , y ′ {\displaystyle y,y'} be vectors in R n {\displaystyle \mathbb {R} ^{n}} . Select an error function E (y , y ′) {\displaystyle E(y,y')} measuring the difference between two outputs. The standard choice is the square of the Euclidean distance between the vectors y {\displaystyle y} and y ′ {\displaystyle y'} : E (y , y ′) = 1 2 ■ y – y ′ ■ 2 {\displaystyle E(y,y')={\tfrac {1}{2}}\lVert y-y'\rVert ^{2}} The error function over n {\textstyle n} training examples can then be written as an average of losses over individual examples: E = 1 2 n ∑ x ■ (y (x) – y ′ (x)) ■ 2 {\displaystyle E={\frac {1}{2n}}\sum _{x}\lVert (y(x)-y'(x))\rVert ^{2}}

6.0 Content Block

To update the weight w i j {\displaystyle w_{ij}} using gradient descent, one must choose a learning rate, η > 0 {\displaystyle \eta >0} . The change in weight needs to reflect the impact on E {\displaystyle E} of an increase or decrease in w i j {\displaystyle w_{ij}} . If ∂ E ∂ w i j > 0 {\displaystyle {\frac {\partial E}{\partial w_{ij}}}>0} , an increase in w i j {\displaystyle w_{ij}} increases E {\displaystyle E} ; conversely, if ∂ E ∂ w i j < 0 {\displaystyle {\frac {\partial E}{\partial w_{ij}}}<0} , an increase in w i j {\displaystyle w_{ij}} decreases E {\displaystyle E} . The new Δ w i j {\displaystyle \Delta w_{ij}} is added to the old weight, and the product of the learning rate and the gradient, multiplied by − 1 {\displaystyle -1} guarantees that w i j {\displaystyle w_{ij}} changes in a way that always decreases E {\displaystyle E} . In other words, in the equation immediately below, − η ∂ E ∂ w i j {\displaystyle -\eta {\frac {\partial E}{\partial w_{ij}}}} always changes w i j {\displaystyle w_{ij}} in such a way that E {\displaystyle E} is decreased:

7.0 Content Block

Consider the network on a single training case: (1 , 1 , 0) {\displaystyle (1,1,0)} . Thus, the input x 1 {\displaystyle x_{1}} and x 2 {\displaystyle x_{2}} are 1 and 1 respectively and the correct output, t is 0. Now if the relation is plotted between the network's output y on the horizontal axis and the error E on the vertical axis, the result is a parabola. The minimum of the parabola corresponds to the output y which minimizes the error E. For a single training case, the minimum also touches the horizontal axis, which means the error will be zero and the network can produce an output y that exactly matches the target output t. Therefore, the problem of mapping inputs to outputs can be reduced to an optimization problem of finding a function that will produce the minimal error. However, the output of a neuron depends on the weighted sum of all its inputs:

8.0 Content Block

The gradient ∇ {\displaystyle \nabla } is the transpose of the derivative of the output in terms of the input, so the matrices are transposed and the order of multiplication is reversed, but the entries are the same:

9.0 Content Block

y {\displaystyle y} is the actual output of the output neuron. In this section, the order of the weight indexes are reversed relative to the prior section: w i j {\displaystyle w_{ij}} is weight from the i {\displaystyle i} th to the

10.0 Content Block

For classification the last layer is usually the logistic function for binary classification, and softmax (softargmax) for multi-class classification, while for the hidden layers this was traditionally a sigmoid function (logistic function or others) on each node (coordinate), but today is more varied, with rectifier (ramp, ReLU) being common.

11.0 Content Block

y {\displaystyle y} : target output For classification, output will be a vector of class probabilities (e.g., (0.1 , 0.7 , 0.2) {\displaystyle (0.1,0.7,0.2)} , and target output is a specific class, encoded by the one-hot/dummy variable (e.g., (0 , 1 , 0) {\displaystyle (0,1,0)}).

12.0 Content Block

After backpropagation During the 2000s it fell out of favour, but returned in the 2010s, benefiting from cheap, powerful GPU-based computing systems. This has been especially so in speech recognition, machine vision, natural language processing, and language structure learning research (in which it has been used to explain a variety of phenomena related to first and second language learning.) Error backpropagation has been suggested to explain human brain event-related potential (ERP) components like the N400 and P600. In 2023, a backpropagation algorithm was implemented on a photonic processor by a team at Stanford University.